



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра математического обеспечения и стандартизации информационных
технологий

Отчет по практической работе №3

по дисциплине «Тестирование и верификация программного обеспечения»

Выполнили:

Студенты группы ИКБО-32-23

Погосян Г.С.

Команда «ВМР»

Ильичев Г.П.

Проверил:

Отчет представлен

«__»_____2025г.

Москва 2025

1. Цель и задачи практической работы

Цель работы состоит в изучении и применении различных подходов к разработке программного обеспечения, основанного на тестировании, для повышения качества, надёжности и поддерживаемости кода.

Для достижения поставленной цели работы студентам необходимо выполнить ряд задач:

- 1) изучить теоретические основы методологий тестирования: TDD, ATDD, BDD и SDD;
- 2) исследовать преимущества и недостатки каждого подхода;
- 3) реализовать практические примеры для каждого метода;
- 4) проанализировать влияние интеграции тестирования на архитектуру и качество программного продукта;
- 5) подготовить итоговый отчёт с выводами и рекомендациями по интеграции подходов в современные процессы разработки.

2. Описание работы

Практическая работа выполняется индивидуально.

В процессе выполнения необходимо разработать программный продукт под Вашим номером, используя TDD, ATDD, BDD и SDD. В отчете необходимо подробно описать все этапы разработки, представленные выше в примерах.

08. Чат-приложение (текстовый мессенджер)

Функции — отправка сообщения, получение сообщения, отображение истории переписки.

TDD — тесты для отправки и получения сообщений, проверка истории.

ATDD — приёмочные тесты для сценариев общения.

BDD — сценарии «Given пользователь А отправил сообщение, When пользователь Б открывает чат, Then сообщение отображается».

SDD — примеры входных сообщений и ожидаемого отображения истории.

3. Чат-приложение (текстовый мессенджер)

1. TDD (Test-Driven Development) — Разработка через тестирование

Суть метода:

В этом подходе разработчик **сначала пишет тесты**, а уже потом — сам код программы, чтобы эти тесты проходили успешно.

Простыми словами:

Сначала мы формулируем, **что программа должна уметь делать** (в виде тестов),

а потом пишем минимальный код, который заставляет тесты пройти.

Процесс:

1. Создаём тесты, описывающие нужное поведение (например, «сообщение должно сохраняться»).
2. Запускаем тесты — они **падают** (так как функции ещё нет).
3. Пишем код, чтобы тест прошёл.
4. Когда тесты проходят — делаем **рефакторинг** (улучшаем код без изменения логики).

Зачем:

Такой подход помогает писать **чистый и надёжный код**, потому что каждая функция сразу проверяется тестом.

```
tdd_tests.py > ...
1 import unittest # Имportsруем библиотеку unittest для написания автоматических тестов
2 from chat_app import send_message, get_conversation, clear_messages # Имportsруем функции, которые будем тестировать
3
4 # Определяем класс тестов, наследуемый от unittest.TestCase
5 class TestChatAppTDD(unittest.TestCase):
6
7     # Метод setUp выполняется перед каждым тестом
8     def setUp(self):
9         clear_messages() # Очищаем список сообщений перед каждым тестом, чтобы тесты были независимыми
10
11     # Тест проверяет, что функция send_message создаёт сообщение с правильной структурой
12     def test_send_message_creates_correct_structure(self):
13         # Отправляем тестовое сообщение и сохраняем результат
14         msg = send_message("Alice", "Bob", "Hello!")
15         # Проверяем, что отправитель указан правильно
16         self.assertEqual(msg["sender"], "Alice")
17         # Проверяем, что получатель указан правильно
18         self.assertEqual(msg["recipient"], "Bob")
19         # Проверяем, что текст сообщения соответствует ожидаемому
20         self.assertEqual(msg["text"], "Hello!")
21         # Проверяем, что в сообщении присутствует ключ "time" (время отправки)
22         self.assertIn("time", msg)
23
24     # Тест проверяет, что get_conversation возвращает сообщения в обе стороны (от Alice к Bob и от Bob к Alice)
25     def test_get_conversation_returns_both_directions(self):
26         # Отправляем сообщение от Alice к Bob
27         send_message("Alice", "Bob", "Hi")
28         # Отправляем сообщение от Bob к Alice
29         send_message("Bob", "Alice", "Hey")
30         # Получаем полную переписку между Alice и Bob
31         conv = get_conversation("Alice", "Bob")
32         # Проверяем, что количество сообщений в переписке равно 2
33         self.assertEqual(len(conv), 2)
34         # Проверяем, что первое сообщение – "Hi"
35         self.assertEqual(conv[0]["text"], "Hi")
36         # Проверяем, что второе сообщение – "Hey"
37         self.assertEqual(conv[1]["text"], "Hey")
38
39     # Если файл запускается напрямую, а не имportsруется, выполняем все тесты
40     if __name__ == "__main__":
41         unittest.main()
42
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Ran 2 tests in 0.000s
OK
```

Фото 1 — метод TDD

2. ATDD (Acceptance Test-Driven Development) — Разработка через приёмочные тесты

Суть метода:

Тесты создаются **вместе с заказчиком или пользователем.**

Они описывают, **как система должна работать с точки зрения клиента.**

Простыми словами:

Перед началом работы все договариваются, **что считается правильной работой программы.**

Эти договорённости превращаются в тесты.

Пример:

Если пользователь отправляет сообщение — оно должно появиться у получателя.

→ Это записывается как тест, и пока программа не выполняет это условие, разработка не считается завершённой.

Зачем:

ATDD помогает избежать недопонимания между разработчиками и заказчиком,

так как требования сразу выражены в виде проверяемых тестов.

```
atdd_tests.py > ...
1 import unittest # Импортируем модуль unittest для написания и запуска тестов
2 from chat_app import send_message, get_conversation, clear_messages # Импортируем функции из chat_app.py, которые будем тестировать
3
4 # Определяем класс тестов, наследуемый от unittest.TestCase
5 class TestChatAppATDD(unittest.TestCase):
6
7     # Метод setUp выполняется перед каждым тестом
8     def setUp(self):
9         clear_messages() # Перед каждым тестом очищаем список сообщений, чтобы начать "с нуля"
10
11     # Тест проверяет, может ли пользователь отправить и получить сообщение
12     def test_user_can_send_and_receive_message(self):
13         # Отправляем сообщение от пользователя "Alice" пользователю "Bob"
14         send_message("Alice", "Bob", "Привет, как дела?")
15
16         # Получаем переписку между Alice и Bob
17         conversation = get_conversation("Alice", "Bob")
18
19         # Проверяем, что в переписке действительно есть сообщение с текстом "Привет, как дела?"
20         self.assertTrue(any(msg["text"] == "Привет, как дела?" for msg in conversation))
21
22     # Если файл запущен напрямую (а не импортирован), выполняем все тесты
23     if __name__ == "__main__":
24         unittest.main()
25
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
-----
Ran 1 test in 0.000s
OK
```

Фото 2 — метод ATDD

3. BDD (Behavior-Driven Development) — Разработка через поведение

Суть метода:

Требования к программе описываются **на простом человеческом языке** (например, английском), в формате сценариев «**Given–When–Then**» (Дано–Когда–Тогда).

Простыми словами:

Мы описываем **поведение системы** так, как его видит пользователь.

Эти описания можно автоматически превратить в тесты.

Пример:

Дано: пользователь "Алиса" отправила сообщение "Привет!" Бобу

Когда: Боб открывает чат

Тогда: он видит сообщение "Привет!"

Зачем:

BDD объединяет разработчиков, тестировщиков и заказчиков, так как сценарии понятны всем и могут автоматически проверяться при запуске тестов.

```
features > ≡ send_receive.feature
1  Feature: Chat communication
2    In order to exchange messages
3    As chat users
4    I want to send and view messages
5
6    Scenario: Viewing a message from another user
7      Given user "Alice" sent message "Hello Bob!" to "Bob"
8      When user "Bob" opens conversation with "Alice"
9      Then he should see message "Hello Bob!"
10
```

Фото 3 — сценарий на языке Gherkin


```
features > steps > bdd_tests.py > step_verify_message
1  from behave import given, when, then
2  from chat_app import send_message, get_conversation, clear_messages
3
4  # Шаг сценария: задано, что пользователь отправил сообщение
5  @given('user "{sender}" sent message "{text}" to "{recipient}"')
6  def step_send_message(context, sender, text, recipient):
7      # Очищаем все предыдущие сообщения перед началом теста
8      clear_messages()
9      # Отправляем сообщение от одного пользователя другому
10     send_message(sender, recipient, text)
11
12 # Шаг сценария: когда пользователь открывает переписку
13 @when('user "{user}" opens conversation with "{other}"')
14 def step_open_conversation(context, user, other):
15     # Получаем текущую переписку между пользователями
16     # и сохраняем её в контекст для последующих шагов
17     context.conversation = get_conversation(user, other)
18
19 # Шаг сценария: тогда пользователь должен увидеть сообщение
20 @then('he should see message "{text}"')
21 def step_verify_message(context, text):
22     # Проверяем, что в переписке есть сообщение с ожидаемым текстом
23     assert any(msg["text"] == text for msg in context.conversation)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
USING RUNNER: behave.runner:Runner
Feature: Chat communication # features/send_receive.feature:1
  In order to exchange messages
  As chat users
  I want to send and view messages
    Then he should see message "Hello Bob!" # features/steps/bdd_tests.py:20 0.000s
    Then he should see message "Hello Bob!" # features/steps/bdd_tests.py:20
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 0.001s
```

Фото 4 — метод BDD

4. SDD (Specification by Example) — Спецификация на примерах

Суть метода:

Требования к системе описываются **на конкретных примерах** (в виде таблиц или наборов данных).

Каждый пример превращается в тест.

Простыми словами:

Вместо длинных описаний — приводим **примеры входных и ожидаемых данных**,

а программа должна вести себя в точности как указано.

Пример таблицы:

Отправитель	Получатель	Сообщение	Ожидаемый результат
Alice	Bob	Привет!	Bob видит "Привет!"

Каждая строка таблицы — это отдельный тест.

Зачем:

SDD помогает убедиться, что программа действительно работает **по конкретным случаям из реальной жизни**, а не только по теоретическим требованиям.

```
test_sdd_tests.py > ...
1 import pytest # Импортируем библиотеку pytest для написания и выполнения тестов
2 from chat_app import send_message, get_conversation, clear_messages # Импортируем функции из chat_app.py
3
4 # Фикстура, которая автоматически выполняется перед каждым тестом
5 @pytest.fixture(autouse=True)
6 def clear():
7     clear_messages() # Очищаем список сообщений перед каждым тестом, чтобы тесты не зависели друг от друга
8
9 # Параметризация теста: одно и то же поведение проверяется с разными данными
10 @pytest.mark.parametrize(
11     "sender, recipient, text, expected_text", # Определяем имена аргументов для теста
12     [
13         ("Alice", "Bob", "Привет!", "Привет!"), # Набор данных №1: Alice → Bob
14         ("Bob", "Alice", "Здравствуй!", "Здравствуй!"), # Набор данных №2: Bob → Alice
15         ("Charlie", "Alice", "Как дела?", "Как дела?"), # Набор данных №3: Charlie → Alice
16     ]
17 )
18 # Основной тест, проверяющий, что сообщения отправляются и сохраняются корректно
19 def test_chat_specifications(sender, recipient, text, expected_text):
20     send_message(sender, recipient, text) # Отправляем сообщение с переданными параметрами
21     conv = get_conversation(sender, recipient) # Получаем переписку между отправителем и получателем
22     # Проверяем, что среди сообщений есть то, в котором текст совпадает с ожидаемым
23     assert any(msg["text"] == expected_text for msg in conv)
24
25 ...
3 passed in 0.01s
```

Фото 5 — метод SSD

Код программы Приложение для флэш-карт (изучение языков)

```
1  from datetime import datetime # Импортируем класс datetime для работы с текущим временем
2
3  messages = [] # Глобальный список для хранения всех сообщений в чате
4
5  # Функция для отправки сообщения от одного пользователя другому
6  def send_message(sender: str, recipient: str, text: str):
7      # Создаём словарь (объект сообщения) с данными об отправителе, получателе, тексте и времени
8      message = {
9          "sender": sender, # Имя отправителя
10         "recipient": recipient, # Имя получателя
11         "text": text, # Текст сообщения
12         "time": datetime.now().strftime("%H:%M:%S") # Текущее время в формате "часы:минуты:секунды"
13     }
14     messages.append(message) # Добавляем сообщение в общий список
15     return message # Возвращаем сообщение, чтобы можно было использовать его при необходимости
16
17 # Функция для получения всей переписки между двумя пользователями
18 def get_conversation(user1: str, user2: str):
19     # Создаём список сообщений, где либо user1 отправил user2, либо наоборот
20     conv = [m for m in messages if
21             (m["sender"] == user1 and m["recipient"] == user2) or
22             (m["sender"] == user2 and m["recipient"] == user1)]
23     return conv # Возвращаем найденную переписку (список сообщений)
24
25 # Функция для очистки всех сообщений (например, перед тестами)
26 def clear_messages():
27     messages.clear() # Очищаем глобальный список сообщений
28
```

Вывод

В ходе выполнения практической работы были изучены и практически применены различные подходы к разработке программного обеспечения, основанные на тестировании: **TDD**, **ATDD**, **BDD** и **SDD**. Каждый из методов продемонстрировал собственные преимущества и особенности применения в процессе создания приложения для флэш-карт.

Метод **TDD** позволил обеспечить высокое качество кода и его устойчивость к ошибкам за счёт предварительного написания тестов. **ATDD** способствовал лучшему пониманию требований со стороны пользователя и формированию прозрачных критериев приёмки. Использование **BDD** сделало процесс тестирования более наглядным и понятным для всех участников разработки, а **SDD** позволил задокументировать функциональность системы в виде конкретных примеров и тестовых случаев.

Применение данных подходов показало, что интеграция тестирования на всех этапах разработки значительно повышает надёжность, предсказуемость и сопровождаемость программного продукта.